# Anexo H: Interfaz gráfica DERMATEC App V6

Este anexo reúne los requisitos de la aplicación, su evolución, la captura de video, las pantallas y las pruebas. La aplicación está escrita en Python 3 con Tkinter (Python Software Foundation, 2024) y usa OpenCV (Bradski, 2000) y Pillow (Clark y Pillow contributors, 2024) para las imágenes. Los dos modelos son archivos TensorFlow Lite con pesos de 16 bits: en Windows se ejecutan con el intérprete de TensorFlow Lite incluido en TensorFlow y en la Raspberry Pi 5 con LiteRT (Google AI Edge, 2025). El resto del código es el mismo en los dos equipos; solo cambia la capa de cámara (ver Anexo L).

## H.1. Requisitos

Los requisitos salieron de la retroalimentación de la dermatóloga sobre las primeras versiones y de las condiciones del sistema embebido (Tabla 1).

**Tabla 1.**

Requisitos de la aplicación y forma en que se atendieron

| Requisito | Origen | Implementación en V6 |
|---|---|---|
| Ver la segmentación y medidas de la lesión | Especialista | Ventanas de segmentación y de parámetros en mm (ver Anexo I) |
| Cargar imágenes además de capturarlas | Especialista | MENÚ → Cargar imágenes (varias a la vez) |
| Historial por paciente y por lesión | Especialista | Carpetas por paciente y lesión; historial de análisis con fecha |
| Seguir la evolución con la regla ABCDE | Especialista | Parámetros guardados con fecha en el historial de la lesión |
| Varias capturas por lesión antes de dar un resultado | Especialista | Resultado agregado de las imágenes de la lesión |
| Observaciones del profesional | Especialista | Campo de observaciones incluido en el informe |
| Rápida y liviana | Sistema embebido | Modelos TFLite de 16 bits; análisis en un hilo aparte |
| Escala y color del equipo | Adquisición | Calibradores en el menú (ver Anexo N) |

Nota. Creado por autores.

La aplicación mide los criterios A, B, C y D. La E (evolución) se apoya en el historial: cada análisis queda guardado con su fecha y sus parámetros, y el profesional compara los informes sucesivos. La aplicación no muestra todavía comparaciones ni tendencias entre análisis; esa función queda como trabajo futuro.

## H.2. Evolución de la aplicación

La interfaz pasó por seis versiones (Tabla 2). Las primeras leían el video con un ciclo de OpenCV en el mismo hilo de Tkinter, de modo que la ventana se congelaba mientras se capturaba o se ejecutaba la red. A partir de la V4 la captura se volvió asíncrona y el análisis

pasó a un hilo aparte. En la V5 se organizó el trabajo por paciente y lesión y se agregó el reporte en PDF. La V6 cambió el modelo y la salida: se retiró la clase “nevo sospechoso”, que resultaba confusa, y el control de calidad dejó de bloquear el resultado (“no concluyente”) para mostrar un aviso, porque en modo bloqueo la mitad de las capturas quedaba sin resultado (50 % en la prueba con capturas del acople).

**Tabla 2.**

Versiones de la aplicación

| Versión | Cambio principal | Problema que resolvió |
|---|---|---|
| V1–V3 | Captura y clasificación en una ventana | Primer prototipo; la interfaz se congelaba |
| V4 | Captura por DirectShow con dos receptores; botón físico SNAP | Video continuo sin bloqueo |
| V5.1–V5.2 | Pantallas de captura y de análisis; pacientes y lesiones; PDF | Trazabilidad por paciente |
| V5.3 | Primer clasificador final (ensamble Z.2 + Z.3) | Mejor desempeño que el modelo multimodal |
| V6 | Ensamble Z.18 + Z.19; salida BENIGNA/MALIGNA + clase; aviso de baja confianza; parámetros en mm; calibración de escala y color; soporte Linux | Uso con el acople y en la Raspberry Pi |

Nota. Creado por autores.

### H.3. Captura de video

En Windows la cámara se abre con DirectShow mediante la biblioteca pygrabber (Schiavinato, 2019). El filtro de captura se abre una sola vez y su señal se envía a dos receptores: uno entrega los cuadros continuos del modo por defecto de la cámara (640 × 480 px en el capilógrafo usado) para la vista en vivo, que se refresca cada 33 ms, es decir, unos 30 cuadros por segundo; el otro atiende el pin de imagen fija que dispara el botón físico SNAP del capilógrafo. Al pulsar el botón se guarda el cuadro en la lesión activa y el receptor se rearma. Al iniciar, la aplicación se conecta sola si encuentra un dispositivo llamado “USB 2.0 Camera”. En los dos sistemas revisa la lista de cámaras cada 3 s y, si el capilógrafo se desconecta y vuelve a aparecer, lo reconecta.

En Linux la cámara se abre por V4L2 con OpenCV, en formato MJPG a 640 × 480 px y con un búfer de un cuadro, para que la vista no se retrase. Aquí la resolución sí se fija por código, porque con ella se calibraron la escala y el color. La lista de dispositivos se lee de /sys/class/video4linux y muestra solo las cámaras USB, una vez por equipo, con su ruta (por ejemplo, “USB Camera (/dev/video0)”). Como el nombre incluye la ruta, la conexión automática al iniciar no ocurre en Linux y el capilógrafo se conecta la primera vez desde DISPOSITIVO.

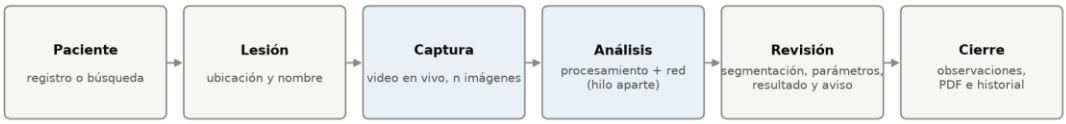
Para el botón físico, la aplicación busca en /proc/bus/input/devices un dispositivo de entrada en la misma ruta USB de la cámara y, si lo encuentra, lo lee en un hilo aparte. Si no lo encuentra, la barra de estado muestra “SNAP físico no detectado (use CAPTURAR o tecla C)”.

### H.4. Pantallas y flujo de trabajo

La aplicación tiene dos pantallas (Figura 1):

1. **Captura.** Vista en vivo, selección del dispositivo, paciente y lesión activos, galería de imágenes capturadas o cargadas y botón ANALIZAR LESIÓN. Cada imagen se guarda en la subcarpeta imagenes de la lesión y se registra en imagenes_manifest.json con su origen, fecha y huella SHA-256, que impide guardar dos veces la misma imagen.
2. **Análisis.** Resultado general de la lesión (BENIGNA o MALIGNA, clase sugerida, probabilidad de malignidad agregada y consistencia entre imágenes), lista de imágenes analizadas con su resultado individual y aviso de baja confianza, y botones para ver la segmentación, ver los parámetros, escribir observaciones, guardar, exportar e imprimir.

**Figura 1.**
Flujo de trabajo en la aplicación




Nota. Creado por autores.

Los modelos no se cargan al abrir la aplicación. Se cargan en el hilo de análisis la primera vez que se pulsa ANALIZAR LESIÓN, de modo que la ventana abre rápido y el video sigue activo mientras los modelos se cargan; los análisis siguientes reutilizan los modelos ya cargados.

El análisis de cada imagen sigue este orden: corrección de color con la calibración del equipo, segmentación y parámetros (Anexo I), clasificación con el ensamble (Anexo K) y control de calidad. La red recibe la captura con la corrección de color aplicada, nunca la imagen sin vello ni la máscara. Los parámetros en milímetros usan la escala del acople activo; mientras esa escala no se haya guardado con el calibrador a partir de la foto de una regla, la aplicación la muestra como “escala ESTIMADA” en ámbar (ver Anexo N).

La imagen recibe el aviso “baja confianza: repetir la captura” si la incertidumbre de la predicción supera el percentil 95 de validación (1,26 bits), si la lesión ocupa más del 60 % del campo o si toca dos o más bordes. El resultado de la lesión promedia las probabilidades de las imágenes que pasan el control; si ninguna pasa, promedia todas y conserva el aviso. Sobre ese promedio se aplican de nuevo el sesgo y el umbral, por lo que el resultado agregado puede diferir del de alguna captura individual. Si después de analizar se agregan o eliminan imágenes, el análisis queda marcado como desactualizado, y la aplicación pide confirmación antes de cerrar o cambiar de lesión con cambios sin guardar.

Los datos se guardan en la carpeta DERMATEC_DATA con una subcarpeta por paciente (paciente.json) y por lesión (lesion.json, imágenes, analisis.json y la carpeta analisis_historial con cada análisis fechado). El informe en PDF es uno por paciente y reúne todas sus lesiones con análisis guardado: portada con los datos del paciente y el resumen de cada lesión, el resultado general de cada lesión, las capturas con su resultado individual, las observaciones del profesional con la localización anatómica y la tabla de parámetros. Lleva la nota “DERMATEC es una herramienta de apoyo para la evaluación inicial de lesiones cutáneas. Los resultados del modelo no sustituyen la valoración ni el diagnóstico médico.” Para exportarlo o imprimirlo, el análisis debe estar guardado.

**Figura 2.**

Pantalla de análisis de la App V6




Nota. Captura de pantalla de la aplicación. Creado por autores.

## H.5. Pruebas de funcionamiento

**Tabla 3.**

Pruebas de la App V6

| Prueba | Criterio | Resultado |
|---|---|---|
| Inicio | Carga de los dos modelos sin congelar la interfaz | Cumple |
| Video en vivo | 640 × 480 px continuo durante el análisis | Cumple (Windows y Raspberry Pi) |
| Botón SNAP | Captura con el botón físico | Cumple en Windows |
| Captura por interfaz | Botón CAPTURAR y tecla C | Cumple |
| Carga de imágenes | Varias imágenes a la vez sin duplicados | Cumple |
| Análisis por imagen (PC) | Color, segmentación, parámetros y ensamble | 0,51 s en promedio (máximo 0,68 s) en 10 capturas con el acople de 8 cm |
| Varias imágenes por lesión | Resultado agregado y aviso por imagen | Cumple |
| Historial | Análisis guardado con fecha por lesión | Cumple |
| Informe PDF | Archivo válido con imágenes, parámetros y observaciones | Cumple |
| Calibración de escala y color | Valores guardados y aplicados a las capturas | Cumple |

Nota. Pruebas en PC con Windows y en Raspberry Pi 5. Creado por autores.

Antes de la versión final se hicieron pruebas de estabilidad sobre la versión V5.2, que fijaron la base de la arquitectura de captura (Tabla 4).

**Tabla 4.**

Pruebas de estabilidad de la versión V5.2

| Prueba | Criterio | Resultado |
|---|---|---|
| Inicio | Carga del modelo y de DirectShow sin congelar la interfaz | Cumple (carga en menos de 2,1 s) |
| Video en vivo | 30 cuadros por segundo a 640 × 480 px sin fugas de memoria | Cumple (más de 4.400 cuadros) |
| Botón SNAP | Recepción de la pulsación y rearme del receptor | Cumple |
| Análisis | Procesamiento y red sin bloquear la interfaz | Cumple (hilo aparte) |
| Informe PDF | Documento con imágenes y resultados | Cumple |

Nota. Versión con el modelo multimodal anterior; los tiempos de análisis de esa versión no son comparables con la V6. Creado por autores.

## H.6. Problemas encontrados y soluciones

Durante el desarrollo de la interfaz se registraron los problemas de la Tabla 5, con su causa y la solución aplicada.

**Tabla 5.**

Problemas de la interfaz y solución

| Problema | Causa | Solución |
|---|---|---|
| La ventana se congelaba al capturar o analizar | Video y red en el mismo hilo de Tkinter | Captura asíncrona y análisis en un hilo aparte |
| Resultado “Nevo, sospechoso, revisar” confuso | Tres salidas posibles para un nevo | Salida BENIGNA o MALIGNA con la clase más probable |
| La mitad de las capturas quedaba “NO CONCLUYENTE” | Control de calidad en modo bloqueo | Control en modo aviso; el resultado se muestra con la advertencia |
| Error al abrir la versión V5.3 | Archivos del modelo en una carpeta distinta | Ruta única MODELO_DERMATEC_FINAL |
| Parámetros vacíos (“—”) en lesiones pequeñas | Se calculaban con una segmentación que descartaba lesiones menores al 2 % del cuadro | Parámetros calculados con la máscara del procesamiento para el especialista |
| Faltaba un módulo al instalar en otro equipo | Archivo de parámetros no incluido en la instalación | Paquete único con todas las dependencias |
| En PowerShell no se activaba el entorno | Política de ejecución de scripts | Ejecución directa con .venv.exe y archivos .bat |
| Lista de cámaras repetida en Linux | V4L2 crea varios nodos por cámara | Filtro por bus USB y una entrada por equipo |

Nota. Creado por autores.

## Referencias


Bradski, G. (2000). The OpenCV library. *Dr. Dobb’s Journal of Software Tools, 25*(11), 120–125.

Clark, A., & Pillow contributors. (2024). *Pillow* [Software]. https://python-pillow.org

Google AI Edge. (2025). *LiteRT* [Software]. https://ai.google.dev/edge/litert

Python Software Foundation. (2024). *tkinter: Python interface to Tcl/Tk* [Documentación]. https://docs.python.org/3/library/tkinter.html

Schiavinato, A. (2019). *pygrabber* [Software]. GitHub. https://github.com/andreaschiavinato/python_grabber